K.R. Mangalam University

School of Engineering and Technology

CAPSTONE PROJECT — PHASE II

Employee Hub: A Go-Based Employee and HR Management Web Application

New Age Programming Languages Lab

Course Code: ENSP465

Submitted by:

Name: Abhinav Jha, Ayush Gupta

Roll Number: 2301010211, 2301010206

Programme: B.Tech CSE

Section: 4/D

Semester: VII

Session: 2026–27

Date: 10 November 2026

Submitted to:

Dr. Hemraj Kumawat

School of Engineering and Technology

K.R. Mangalam University

GitHub Repository: Employee Hub

GitHub URL: https://github.com/Abh1nav90/Employee-Hub

1. Project Overview

Employee Hub is a browser-based employee and human-resource management application. It brings employee information and selected HR operations into one interface. Users can navigate between the employee directory, departments, attendance, leave management and payroll screens.

The application uses Go for its backend and HTML, CSS and JavaScript for the browser interface. The submitted project contains Go source packages, web assets and local JSON data files. It is designed as a local academic prototype and does not require an external database URL or hosted database service.

2. Project Objectives

- Maintain employee and department information in a central interface.
- Present workforce summaries through a dashboard.
- Provide dedicated screens for attendance, leave requests and payroll records.
- Organize source code into models, services, utilities, patterns and algorithms.
- Use local JSON files for sample records and local execution.
- Document the implemented application and demonstrate its user interface through screenshots.

3. Key Features and Scope

Module	Key functionality shown in the application
Dashboard	Workforce summary cards for total employees, active workforce, departments and average salary, with department and workforce-status summaries.
Employee Directory	Employee list with department, role, location, employment type and status; search, filters and record-action controls are visible.
Departments	Department cards for Engineering, Finance and HR, with employee counts, active counts, salary summaries and employee listings.
Attendance	Attendance summary and recent records showing date, employee, status, check-in and check-out times.

Leave Management	Leave request totals and statuses, including a pending request with Approve and Reject controls.
Leave Request Form	Form fields for employee, leave type, start date, end date and reason.
Payroll	Salary register with two sample payslip entries and payroll total and average summaries.

The sidebar also lists Onboarding, Organization, Performance, Calendar, Help desk, Documents, Reports, Activity and Settings. These are additional navigation areas; their complete workflows should be described as implemented only after they have been tested.

4. Technologies Used

Technology	Purpose
Go	Main backend language and application runtime.
HTML	Structure of browser pages and forms.
CSS	Layout, styling and visual presentation.
JavaScript	Browser-side interactions and dynamic interface behaviour.
JSON	Local storage format for sample employee and HR records.
Go standard library	HTTP request handling and JSON processing where used in the source.

5. Implementation and Project Structure

The application follows a browser-to-server structure. The browser presents pages, tables and forms. The Go application handles server-side operations and works with structured records. Supporting code is separated into packages, while the data directory contains JSON files for local records.

File or folder	Purpose
main.go and go.mod	Application entry point and Go module configuration.

models/	Data model definitions.
services/	Service-level operations, including employee-related logic.
utils/	Helper functions, including JSON-related utilities.
algorithms/	Search and sorting-related source files.
patterns/	Reusable pattern-related code.
web/	HTML, CSS and JavaScript interface files.
data/	Local JSON records for employee and HR modules.

5.1 System Architecture

EmployeeHub uses a browser-to-server structure. The user interacts with the web interface, which communicates with the Go application. The Go application processes the requested operation and uses the relevant model, service or utility code. Local JSON files provide the records used by the application.

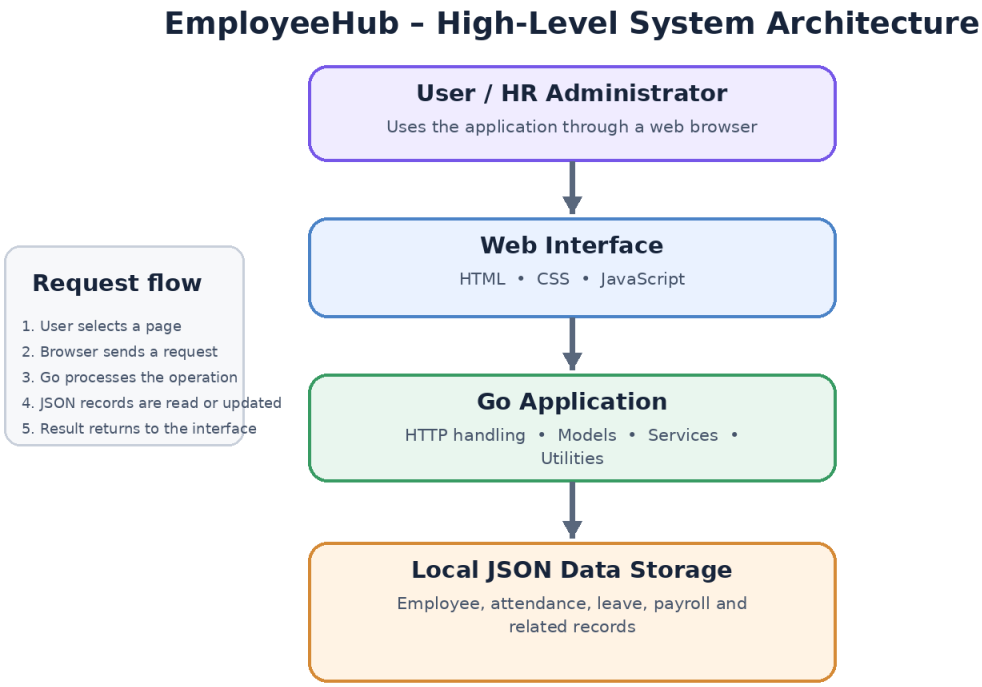


Figure: High-level architecture of the Employee Hub application.

5.2 Local Execution

The project README provides the following startup procedure: open a terminal in the folder containing go.mod, run the command `go run .`, and open `http://localhost:8080` in a browser. Follow the README instructions if the default port is already in use.

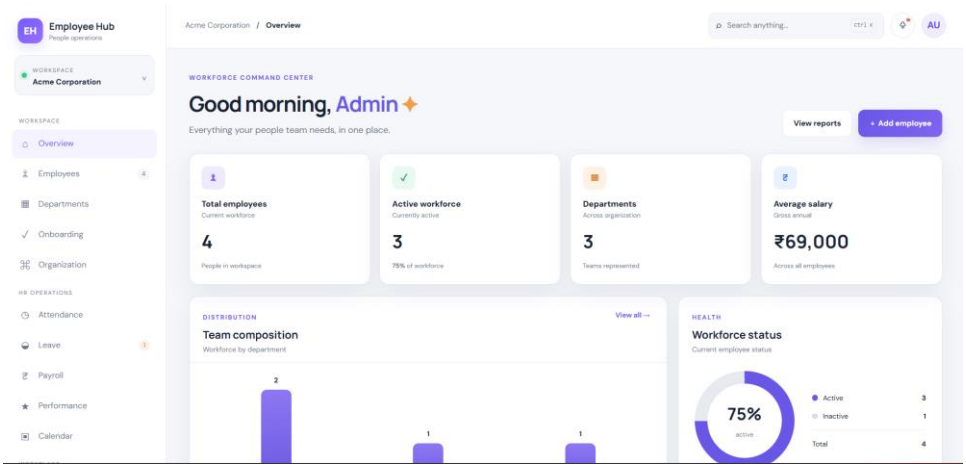
5.3 Data Storage

The submitted project includes JSON files for attendance, documents, employees, events, leaves, onboarding, payroll, performance and tickets. The README also describes the use of a notifications JSON file. Local JSON storage keeps the prototype simple to run without configuring an external database.

6. User Interface Screenshots

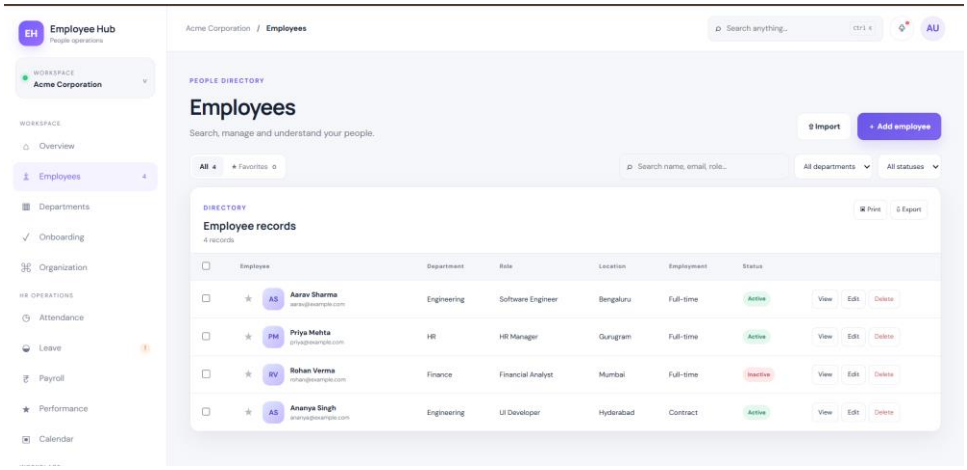
The following screenshots are for the Phase-II report. They demonstrate the visible application screens and sample records.

6.1 Dashboard



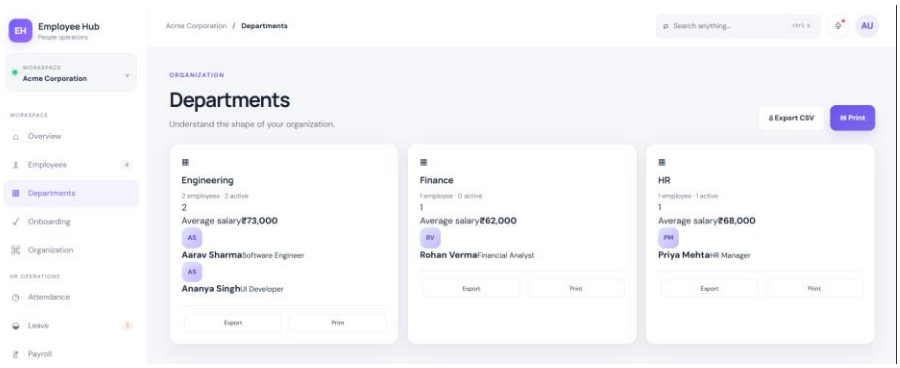
Shows 4 total employees, 3 active employees, 3 departments and an average salary of ₹69,000, with department and workforce-status summaries.

6.2 Employee Directory



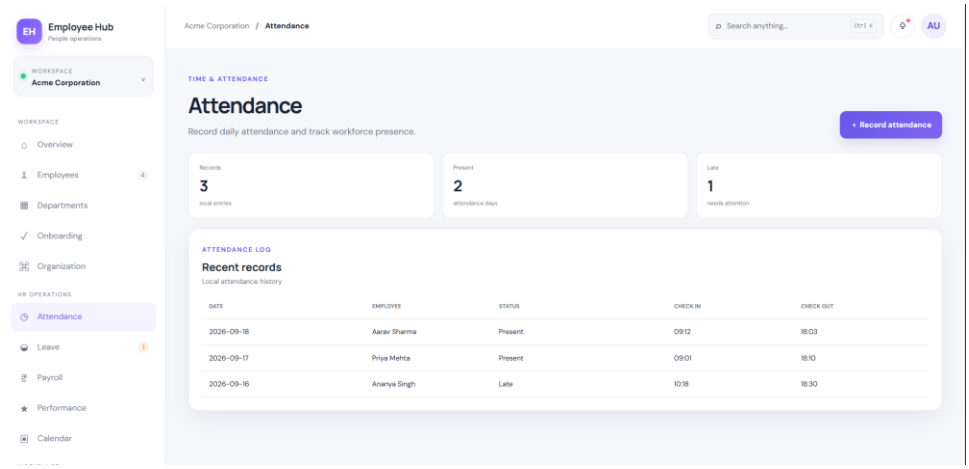
Lists four sample employees with department, role, location, employment type and status. Search, filters and record-action controls are visible.

6.3 Departments



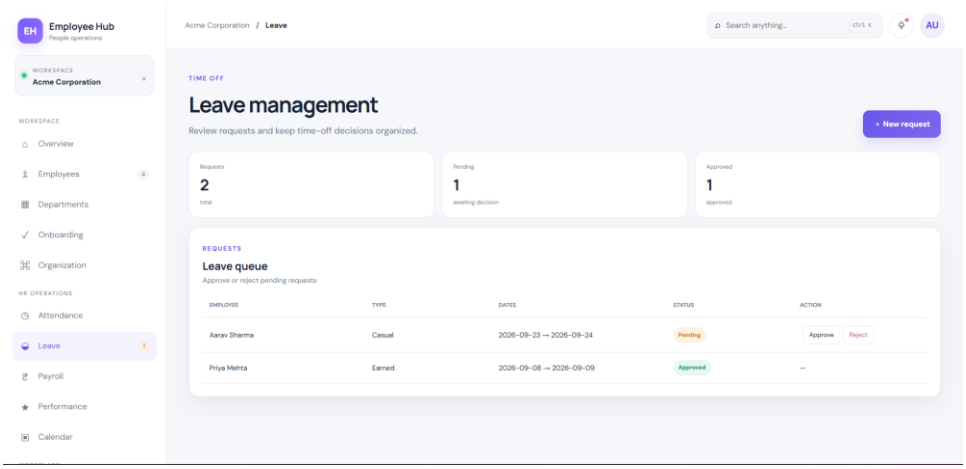
Shows Engineering, Finance and HR department cards, employee counts, active counts, average salary figures and employee listings.

6.4 Attendance



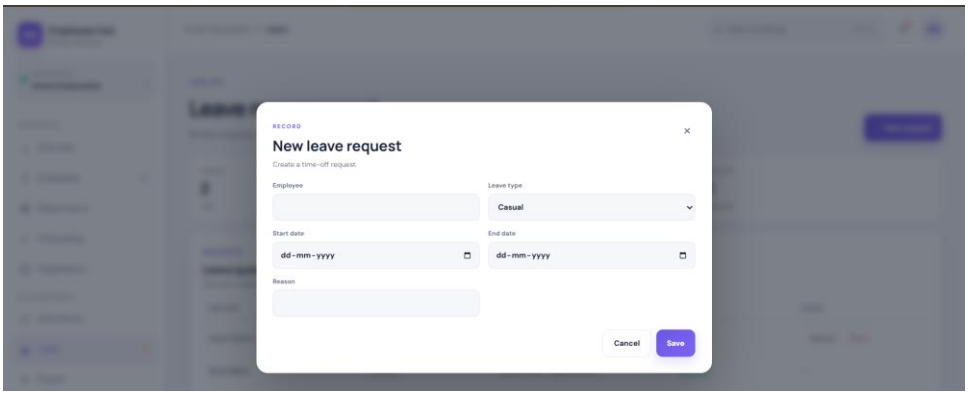
Shows three attendance records: two Present and one Late, with dates, employee names and check-in/check-out times.

6.5 Leave Management



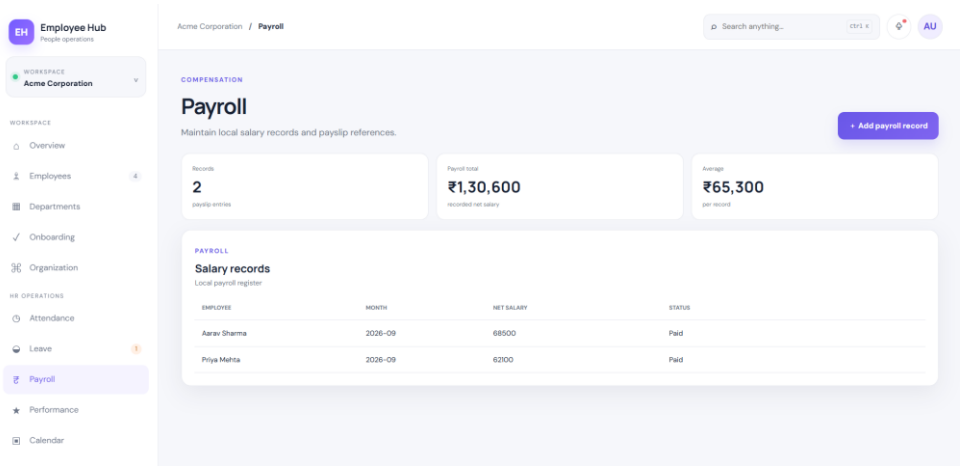
Shows two leave requests, one Pending and one Approved, with Approve and Reject controls for the pending request

6.6 Leave Request Form



Shows the leave request fields: employee, leave type, start date, end date and reason.

6.7 Payroll



Shows two September 2026 payslip entries, a recorded payroll total of ₹1,30,600 and an average net salary of ₹65,300.

6.8 Other Modules



Shows navigation links for Onboarding, Organization, Performance, Calendar, Help desk, Documents, Reports, Activity and Settings. Navigation visibility alone does not verify each workflow.

7. Testing and Verification

The Go packages in the submitted source archive were checked using `go test ./...`. The command completed successfully, but the output showed [no test files] for the packages. This confirms that the Go packages compile; it does not establish automated test coverage for all application workflows.

The screenshots demonstrate the appearance of selected pages, but they do not by themselves prove that every button or form works. Before the final demonstration, manually verify application startup, employee search and filters, leave request submission and approval/rejection, attendance and payroll records, and any export or print actions that will be demonstrated.

8. Limitations and Future Improvements

Employee Hub is a local academic prototype using JSON files rather than a production database.

Possible improvements include adding automated tests, strengthening authentication and authorization, improving safe concurrent data updates, adding backup and restore support, and testing and documenting the additional modules.

9. Conclusion

Employee Hub demonstrates how Go can be used to build a browser-based application for employee information and selected HR operations. Its source structure separates backend code, browser assets and local JSON data. The supplied screenshots document the main visible screens, and the project provides a foundation for further improvements in testing, security and module completeness.

10. GitHub Repository Link

GitHub Repository: Employee Hub

GitHub URL: https://github.com/Abh1nav90/Employee-Hub